Ex. No. 07	APPLICATION STATE MANAGEMENT AND SERIALIZATION
Date of Exercise	
Git Hub Host Link	

Aim

To manage application state and serialize/deserialize data using JSON in a Flutter application.

Description

This experiment demonstrates application state management using `setState()` and JSON serialization/deserialization using a `Student` model. Student data is converted from a Dart object into JSON format using `toJson()` and converted back into a Dart object using `fromJson()`. A list of students is displayed dynamically using `ListView.builder()`.

Procedure:

1. A new Flutter project named `state_app` was created using the Flutter command.
2. A Dart file named `student.dart` was created inside the `lib` folder to define the `Student` model.
3. The `Student` class was created with the properties `id`, `name`, and `course`.
4. The `toJson()` method was implemented to convert the Student object into a JSON-compatible `Map`.
5. The `fromJson()` factory constructor was implemented to convert JSON data back into a Student object.
6. A Student object was created with the values 1, Ragu, and CSE.
7. The `toJson()` method was used to serialize the Student object into JSON data.
8. The `fromJson()` method was used to deserialize the JSON data back into a Student object.
9. A `List<String>` was created to maintain the student names as application state.
10. The `setState()` method was used to add a student to the list whenever the Add Student button was pressed.
11. `ListView.builder()` was used to dynamically display the student names.
12. The application was executed and tested in the Chrome browser.

Program

student.dart

```
class Student {
  final int id;
  final String name;
  final String course;
  Student({
    required this.id,
    required this.name,
    required this.course,
  });
  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'name': name,
      'course': course,
    };
  }
  factory Student.fromJson(Map<String, dynamic> json) {
    return Student(
      id: json['id'],
      name: json['name'],
      course: json['course'],
```

```
    );
  }
}
```

main.dart

```
import 'package:flutter/material.dart';
import 'student.dart';

void main() {
  runApp(const StateApp());
}

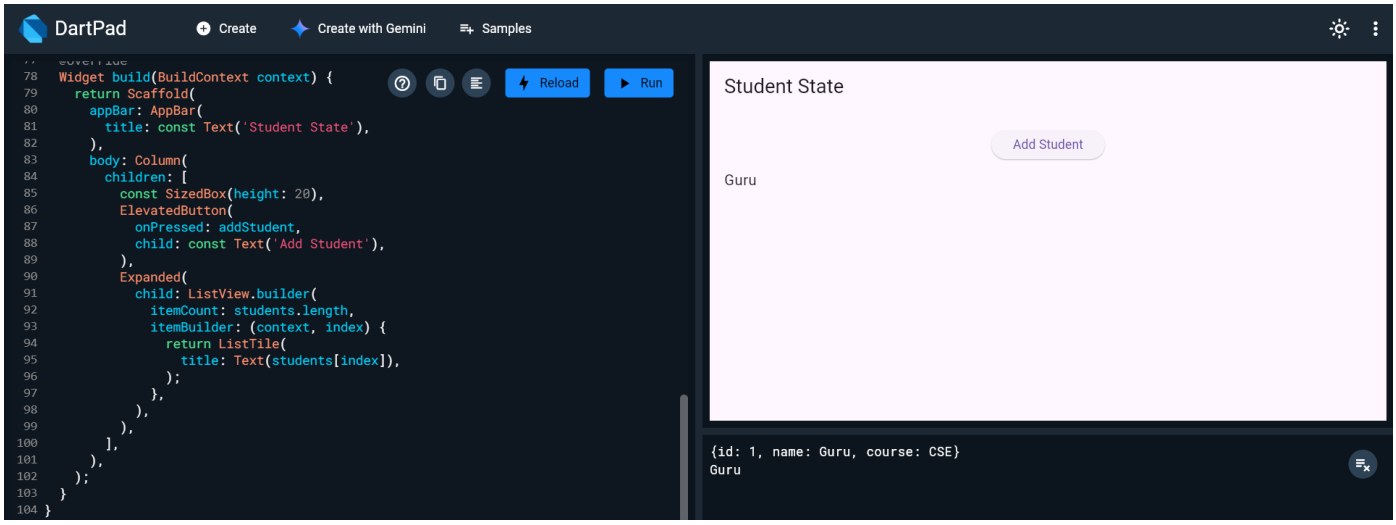
class StateApp extends StatelessWidget {
  const StateApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
    debugShowCheckedModeBanner: false,
    title: 'State Management',
      home: const StudentPage(),
    );
  }
}
```

```
class StudentPage extends StatefulWidget {
  const StudentPage({super.key});
  @override
  State<StudentPage> createState() => _StudentPageState();
}
class _StudentPageState extends State<StudentPage> {
  List<String> students = [];
  Student student = Student(
    id: 1,
    name: 'Guru',
    course: 'CSE',
  );
  @override
  void initState() {
    super.initState();
    Map<String, dynamic> data = student.toJson();
    print(data);
    Student newStudent = Student.fromJson(data);
    print(newStudent.name);
  }
  void addStudent() {
    setState(() {
    students.add('Guru');
    });
  }
```

```
@override
Widget build(BuildContext context) {
 return Scaffold(
  appBar: AppBar(
   title: const Text('Student State'),
  ),
  body: Column(
   children: [
    const SizedBox(height: 20),
    ElevatedButton(
     onPressed: addStudent,
     child: const Text('Add Student'),
    ),
    Expanded(
     child: ListView.builder(
      itemCount: students.length,
      itemBuilder: (context, index) {
      return ListTile(
         title: Text(students[index]),
       );
      },
     ),
    ),
   ],
  ),
```

Output



Result

Thus, the Flutter application was successfully developed to manage changing student data using `setState()`. The Student object was successfully serialized into JSON and deserialized back into a Dart object. The student data was also dynamically displayed using `ListView.builder()`.